



Guía del Estudiante — Seguridad en el Pipeline CI con SonarCloud y Snyk

Asignatura DevOps | Actividad 2.4 — Añadiendo seguridad a nuestro Pipeline

DUOC UC · DevOps 003D

JaCoCo · JUnit 5

SonarCloud

Snyk

GitHub Actions

¿Qué vas a lograr? Integrar dos herramientas profesionales de seguridad y calidad — **SonarCloud** y **Snyk** — en el pipeline de GitHub Actions del proyecto Spring Boot. Al finalizar, tu pipeline ejecutará tests, medirá cobertura con JaCoCo, analizará la calidad del código con SonarCloud y detectará vulnerabilidades en dependencias con Snyk, todo de forma automatizada.



Paso 0 — Verifica tu entorno

Antes de comenzar confirma que tienes las herramientas de la actividad anterior:

Herramienta	¿Cómo verificar?	Necesario para
Java JDK 17	<code>java -version</code>	Compilar y testear
Maven 3.8+	<code>mvn -version</code>	Ejecutar tests y análisis
Git	<code>git --version</code>	Publicar cambios al pipeline
Docker Desktop o Podman	<code>docker --version</code> / <code>podman --version</code>	Build de imagen al final

Además necesitarás **cuentas gratuitas** en:

Plataforma	URL	¿Para qué?
SonarCloud	https://sonarcloud.io	Análisis de calidad de código
Snyk	https://snyk.io	Detección de vulnerabilidades
GitHub	https://github.com	Repositorio y pipeline (Actions)

💡 Las tres plataformas tienen planes gratuitos para repositorios públicos. Regístrate con tu cuenta de GitHub para simplificar la integración.



Paso 1 — Punto de partida: el proyecto de la actividad anterior

Esta actividad **parte directamente del proyecto** de la Actividad 2.3. Debes tener:

```
bdget/
├── .github/
│   └── workflows/
│       └── main.yml      ← pipeline que vas a extender
├── src/
│   ├── main/java/...    ← código fuente
│   └── test/java/...    ← pruebas unitarias ya escritas
└── pom.xml              ← JaCoCo ya configurado
```

Verifica que el pipeline de la actividad anterior sigue funcionando:

```
git log --oneline -3      # confirma que tienes commits previos
mvn test                  # confirma que los tests pasan localmente
```

🔍 Paso 2 — ¿Qué son SonarCloud y Snyk?

SonarCloud

Calidad de código

SonarCloud analiza tu código fuente en busca de:

Categoría	Ejemplos
Bugs	Variables no inicializadas, condiciones siempre verdaderas
Code Smells	Métodos demasiado largos, código duplicado, complejidad excesiva
Vulnerabilidades	Inyección SQL, contraseñas hardcodedas en código fuente
Cobertura	Lee el reporte de JaCoCo e integra el porcentaje en su dashboard
Duplicación	Bloques de código copiados entre clases

SonarCloud asigna una nota de **A a E** a cada categoría y puede **bloquear** un pull request si la calidad no cumple un umbral mínimo.

Snyk

Vulnerabilidades

Snyk analiza las dependencias declaradas en tu `pom.xml` y las capas de la imagen Docker:

Lo que analiza	¿Dónde busca?
Dependencias Maven	<code>pom.xml</code> → CVEs conocidos en cada librería
Imagen Docker	<code>Dockerfile</code> → vulnerabilidades en paquetes del SO base
Severidad	Critical, High, Medium, Low según la base CVSS

⚙️ Paso 3 — Configura SonarCloud

3a — Crea el proyecto en SonarCloud

1. Ingresa a <https://sonarcloud.io> con tu cuenta de GitHub
2. Click en "+" → "Analyze new project"
3. Selecciona tu repositorio `bdget` y click en "Set Up"
4. Elige "With GitHub Actions" como método de análisis
5. SonarCloud te mostrará:
 - Tu `SONAR_TOKEN` — cópialo, lo usarás en el Paso 5
 - Tu `SONAR_PROJECT_KEY` — anótalo (formato: `tu-usuario_bdget`)
 - Tu `SONAR_ORGANIZATION` — anótalo (tu nombre de usuario en SonarCloud)

3b — Agrega las propiedades al pom.xml

Abre `pom.xml` y agrega dentro de `<properties>`:

```
<properties>
  <java.version>17</java.version>
  <!-- SonarCloud -->
  <sonar.projectKey>TU-USUARIO_bdget</sonar.projectKey>
  <sonar.organization>TU-USUARIO-SONARCLOUD</sonar.organization>
  <sonar.host.url>https://sonarcloud.io</sonar.host.url>
</properties>
```

⚠ Reemplaza `TU-USUARIO_bdget` y `TU-USUARIO-SONARCLOUD` con los valores que SonarCloud te mostró en el paso anterior.

🔧 Paso 4 — Configura Snyk

4a — Obtén tu token de Snyk

1. Ingresa a <https://snyk.io> con tu cuenta de GitHub
2. Click en tu avatar (arriba derecha) → **"Account settings"**
3. En la sección **"Auth Token"** → click en **"click to show"**
4. Copia el token — lo usarás como Secret de GitHub

4b — (Opcional) Instala el CLI de Snyk localmente

Si quieres probar Snyk antes de ejecutarlo en el pipeline:

```
# Con npm (si tienes Node.js instalado)
npm install -g snyk

# Autentícate
snyk auth

# Escanea las dependencias del proyecto
snyk test --all-projects
```

🔑 Paso 5 — Configura todos los Secrets en GitHub

Ve a tu repositorio → **Settings** → **Secrets and variables** → **Actions** → **"New repository secret"**

Secret	Dónde obtenerlo
<code>SONAR_TOKEN</code>	SonarCloud → tu proyecto → Administration → Analysis Method → GitHub Actions
<code>SONAR_PROJECT_KEY</code>	SonarCloud → tu proyecto → Information (ej: <code>miusuario_bdget</code>)
<code>SONAR_ORGANIZATION</code>	SonarCloud → tu nombre de organización (ej: <code>miusuario</code>)
<code>SNYK_TOKEN</code>	Snyk → Account Settings → Auth Token
<code>DOCKERHUB_USERNAME</code>	Ya configurado en actividad anterior
<code>DOCKERHUB_TOKEN</code>	Ya configurado en actividad anterior

✅ Los secrets `DOCKERHUB_USERNAME` y `DOCKERHUB_TOKEN` ya existen de la Actividad 2.3 — no necesitas recrearlos.

Paso 6 — Actualiza el pipeline: agrega SonarCloud y Snyk

Reemplaza el contenido de `.github/workflows/main.yml` con el siguiente pipeline completo:

```
name: CI - Tests, Calidad y Seguridad

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-test-analyze:
    runs-on: ubuntu-latest

    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: root_pass
          MYSQL_DATABASE: bdget_db
          MYSQL_USER: bdget_user
          MYSQL_PASSWORD: bdget_pass
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping -h localhost -u root -proot_pass"
          --health-interval=10s --health-timeout=5s --health-retries=5

    steps:
      # 1. Descargar el código (fetch-depth:0 requerido por SonarCloud)
      - name: Checkout del repositorio
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      # 2. Configurar Java 17 con caché Maven
      - name: Configurar Java 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven

      # 3. Ejecutar tests + generar reporte JaCoCo
      - name: Ejecutar tests
        run: mvn test
        env:
          SPRING_DATASOURCE_URL: jdbc:mysql://localhost:3306/bdget_db?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval
            =true
          SPRING_DATASOURCE_USERNAME: bdget_user
          SPRING_DATASOURCE_PASSWORD: bdget_pass

      # 4. Subir reporte JaCoCo como artefacto
      - name: Subir reporte JaCoCo
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/

      # 5. Analizar con SonarCloud
      - name: Análisis SonarCloud
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
        run: |
          mvn sonar:sonar \
            -Dsonar.projectKey=${ secrets.SONAR_PROJECT_KEY } \
            -Dsonar.organization=${ secrets.SONAR_ORGANIZATION } \
```

```

-Dsonar.host.url=https://sonarcloud.io \
-Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml

# 6. Escanear dependencias Maven con Snyk
- name: Escaneo de vulnerabilidades Snyk (dependencias)
  uses: snyk/actions/maven@master
  continue-on-error: true
  env:
    SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
  with:
    args: --severity-threshold=high

# 7. Compilar el .jar
- name: Build del proyecto
  run: mvn package -DskipTests

# 8. Login en DockerHub
- name: Login en DockerHub
  uses: docker/login-action@v3
  with:
    username: ${ secrets.DOCKERHUB_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }

# 9. Construir imagen Docker
- name: Build de la imagen Docker
  run: docker build -t ${ secrets.DOCKERHUB_USERNAME }/bdget-app:latest .

# 10. Escanear imagen Docker con Snyk
- name: Escaneo de imagen Docker con Snyk
  uses: snyk/actions/docker@master
  continue-on-error: true
  env:
    SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
  with:
    image: ${ secrets.DOCKERHUB_USERNAME }/bdget-app:latest
    args: --severity-threshold=high

# 11. Publicar imagen en DockerHub
- name: Push de la imagen a DockerHub
  run: docker push ${ secrets.DOCKERHUB_USERNAME }/bdget-app:latest

```

Paso 7 — Entiende cada nuevo paso del pipeline

fetch-depth: 0 — ¿Por qué es obligatorio para SonarCloud?

```

- uses: actions/checkout@v4
  with:
    fetch-depth: 0    # descarga TODO el historial de commits

```

SonarCloud necesita el historial completo para calcular métricas de "código nuevo" vs "código existente" y aplicar reglas de Quality Gate solo al código modificado. Sin esto, el análisis falla o es incorrecto.

mvn sonar:sonar — ¿Qué envía a SonarCloud?

```

mvn sonar:sonar \
-Dsonar.projectKey=TU-USUARIO_bdget \
-Dsonar.organization=TU-USUARIO \
-Dsonar.host.url=https://sonarcloud.io \
-Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml

```

Este comando envía el código + el reporte de cobertura a SonarCloud. El parámetro `jacoco.xml` le dice a Sonar dónde encontrar la cobertura generada en el paso 3.

 ¿Por qué `jacoco.xml` y no `jacoco.exec`? SonarCloud lee el reporte en formato XML. JaCoCo genera ambos: `.exec` (binario interno) y `jacoco.xml` dentro de `target/site/jacoco/`. El plugin sonar lee el XML.

Snyk en dependencias Maven

```
- uses: snyk/actions/maven@master
  continue-on-error: true      # no bloquea el pipeline
  with:
    args: --severity-threshold=high  # solo High y Critical
```

`snyk/actions/maven` lee el `pom.xml` y consulta la base de datos de CVEs de Snyk. `continue-on-error: true` permite que el pipeline continúe incluso si encuentra vulnerabilidades.

 En entornos de producción se quitaría `continue-on-error: true` para **bloquear** el pipeline ante vulnerabilidades críticas.

Snyk en imagen Docker

```
- uses: snyk/actions/docker@master
  with:
    image: usuario/bdget-app:latest
    args: --severity-threshold=high
```

Snyk escanea los paquetes del sistema operativo instalados en la imagen base (`eclipse-temurin:17-jre`) buscando CVEs conocidos. Una imagen base desactualizada puede tener vulnerabilidades aunque tu código sea perfecto.

 Paso 8 — Interpreta los resultados en SonarCloud

Después de que el pipeline ejecute el paso de SonarCloud, ve a <https://sonarcloud.io> y abre tu proyecto:

Quality Gate:  PASSED (o  FAILED)

Bugs	Vulner.	Smells	Coverage	Duplication
0 A	0 A	5 B	78.5%	2.1%

¿Qué es el Quality Gate?

El Quality Gate es un conjunto de condiciones mínimas que tu código debe cumplir. El Quality Gate por defecto de SonarCloud ("Sonar way") requiere que el **código nuevo**:

Condición	Umbral
Sin bugs nuevos	0
Sin vulnerabilidades nuevas	0
Cobertura en código nuevo	≥ 80%
Duplicación en código nuevo	< 3%

Navega por el reporte

- 1. Click en **"Issues"** — lista todos los code smells, bugs y vulnerabilidades

2. Click en un issue — muestra la línea exacta del código con explicación y cómo corregirlo
3. Click en "**Coverage**" — muestra el reporte visual de JaCoCo integrado en la web

Paso 9 — Interpreta los resultados en Snyk

Resultados en GitHub Actions

En la pestaña **Actions** de GitHub, expande el paso "**Escaneo de vulnerabilidades Snyk**":

```
Testing bdget...

x High severity vulnerability found in com.oracle.database.jdbc:ojdbc11
  Description: Improper Input Validation
  Info: https://security.snyk.io/vuln/SNYK-JAVA-...
  Introduced through: com.oracle.database.jdbc:ojdbc11@21.x.x
  Fixed in: 21.y.y (si existe versión corregida)

Tested 42 dependencies for known issues, found 1 issue.
```

Resultados en el dashboard de Snyk

1. Ingresa a <https://snyk.io> → **Projects**
2. Verás tu repositorio `bdget` listado con el número de vulnerabilidades
3. Click en el proyecto → verás el detalle de cada CVE

Campo	Significado
Severity	Critical / High / Medium / Low
CVE	Identificador oficial (ej: CVE-2023-XXXXX)
Introduced through	Cuál dependencia de tu <code>pom.xml</code> lo trae
Fixed in	Versión donde fue corregido (si existe)
Is upgradeable	Si Snyk puede aplicar el fix automáticamente

Paso 10 — Ejecuta el pipeline completo

Una vez que tienes todos los secrets configurados y el YAML actualizado:

```
git add .github/workflows/main.yml pom.xml
git commit -m "feat: integra SonarCloud y Snyk en el pipeline CI"
git push origin main
```

Ve a la pestaña **Actions** de tu repositorio y observa la ejecución:

✓	Checkout del repositorio	~5s	
✓	Configurar Java 17	~15s	
✓	Ejecutar tests	~60s	
✓	Subir reporte JaCoCo	~5s	→ artefacto disponible
✓	Análisis SonarCloud	~90s	→ resultados en sonarcloud.io
✓	Escaneo Snyk (dependencias)	~30s	→ resultados en snyk.io
✓	Build del proyecto	~30s	
✓	Login en DockerHub	~5s	
✓	Build de la imagen Docker	~120s	
✓	Escaneo Snyk (imagen Docker)	~45s	→ resultados en snyk.io
✓	Push de la imagen a DockerHub	~30s	

🕒 **El pipeline completo tarda ~7-10 minutos** la primera vez por las descargas. Las siguientes ejecuciones son más rápidas gracias al caché de Maven.

🔴 Solución de problemas frecuentes

SonarCloud falla con "Project not found"

Verifica que el `SONAR_PROJECT_KEY` en el pipeline coincide exactamente con el que muestra SonarCloud (sensible a mayúsculas y guiones bajos).

```
# Formato típico: organizacion_nombre-repo
# Ejemplo:      miusuario_bdget
```

SonarCloud falla — `fetch-depth` incorrecto

Asegúrate de que el step de checkout tiene `fetch-depth: 0` exactamente:

```
- uses: actions/checkout@v4
  with:
    fetch-depth: 0
```

Snyk falla con "Missing SNYK_TOKEN"

Verifica que el secret `SNYK_TOKEN` está configurado en GitHub y que el nombre en el YAML coincide exactamente (sensible a mayúsculas).

Snyk reporta muchas vulnerabilidades en la imagen Docker

La imagen base `eclipse-temurin:17-jre` puede tener vulnerabilidades en paquetes del SO. Usa la variante Alpine que tiene menos paquetes:

```
# En Dockerfile, reemplaza:
FROM eclipse-temurin:17-jre
# Por la variante Alpine (menos CVEs):
FROM eclipse-temurin:17-jre-alpine
```

Quality Gate de SonarCloud falla por cobertura baja

SonarCloud requiere $\geq 80\%$ de cobertura en código nuevo. Revisa el reporte local y agrega más tests:

```
mvn test
open target/site/jacoco/index.html # identifica clases con baja cobertura
# agrega tests → git push
```

Resumen de comandos

Acción	Comando
Ejecutar tests + cobertura	<code>mvn test</code>
Ejecutar análisis Sonar local	<code>mvn sonar:sonar -Dsonar.token=TU_TOKEN</code>
Escanear dependencias con Snyk CLI	<code>snyk test</code>
Escanear imagen Docker con Snyk CLI	<code>snyk container test usuario/bdget-app:latest</code>
Ver reporte JaCoCo local (macOS)	<code>open target/site/jacoco/index.html</code>
Limpiar y recompilar todo	<code>mvn clean test</code>
Publicar cambios al pipeline	<code>git add . && git commit -m "msg" && git push</code>

Lista de verificación — entregables

Al terminar la actividad debes tener evidencia de cada punto:

- ☐ Captura del pipeline en GitHub Actions con **todos los pasos en verde** 
- ☐ Captura del **dashboard de SonarCloud** mostrando el Quality Gate y las métricas (Bugs, Vulnerabilities, Coverage)
- ☐ Captura del reporte de **Issues en SonarCloud** (aunque sea con 0 issues)
- ☐ Captura del **reporte de Snyk** en GitHub Actions — resultado del escaneo de **dependencias Maven**
- ☐ Captura del **reporte de Snyk** en GitHub Actions — resultado del escaneo de la **imagen Docker**
- ☐ Captura de la sección **Artifacts** en GitHub Actions con el artefacto `jacoco-report` disponible
- ☐ Captura de los **Secrets configurados** en GitHub (Settings → Secrets → Actions) — sin mostrar los valores